

Dimensionality Reduction for NLP

Lecture 10: PCA, t-SNE, and UMAP

PSYC 51.07: Models of Language and Communication - Week 4
Winter 2026

Today's Lecture

1.  **The Curse of Dimensionality**
2.  **Principal Component Analysis (PCA)**
3.  **t-SNE: Stochastic Neighbor Embedding**
4.  **UMAP: Uniform Manifold Approximation**
5.  **Comparison & Best Practices**
6.  **Visualization Techniques**

Goal: Learn to visualize and explore high-dimensional embeddings

The Curse of Dimensionality

The Problem:

Word Embeddings:

- Word2Vec: 300 dimensions
- GloVe: 300 dimensions
- FastText: 300 dimensions
- BERT: 768 dimensions
- GPT-3: 12,288 dimensions!

Challenges:

- Can't visualize 300D space
- Distances behave strangely
- Computation expensive
- Storage requirements
- Interpretation difficult

Interesting Properties:

- In high dimensions, most points are far apart
- Volume concentrates in corners
- Random vectors nearly orthogonal
- Nearest neighbors may not be near!

Example

In 300D space with uniform distribution:

- Expected distance to nearest neighbor: $\approx$ distance to farthest!
- All points approximately equidistant

Solution: Dimensionality Reduction

Why Dimensionality Reduction?

Visualization:

- 2D/3D plots
- Explore semantic structure
- Identify clusters
- Discover patterns
- Quality assurance

Computation:

- Faster algorithms
- Less memory
- Enable real-time systems
- Scalability

Machine Learning:

- Reduce overfitting
- Feature selection
- Improve generalization
- Remove noise
- Combat curse of dimensionality

Interpretation:

- Understand relationships
- Identify important dimensions
- Communicate results
- Debug models

The Goal

Reduce from d dimensions (e.g., 300) to k dimensions (e.g., 2 or 3) while **preserving important structure**

Principal Component Analysis (PCA)



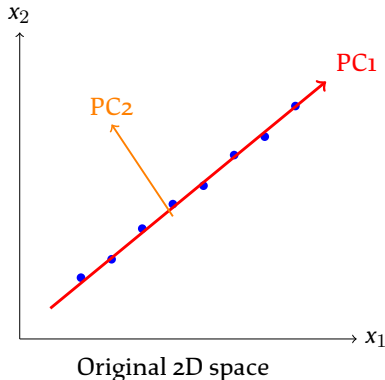
The classic linear dimensionality reduction method

Intuition:

- Find directions of maximum variance
- Project data onto these directions
- First PC: most variance
- Second PC: second most (orthogonal)
- And so on...

Properties:

- Linear transformation
- Preserves global structure
- Deterministic
- Fast computation
- Interpretable (sometimes)



PCA: The Algorithm

Mathematical formulation:

1. Center the data:

- Subtract mean: $\tilde{X} = X - \bar{X}$
- Each dimension has mean 0

2. Compute covariance matrix:

- $C = \frac{1}{n-1} \tilde{X}^T \tilde{X}$
- Captures correlations between dimensions

3. Eigenvalue decomposition:

- $C = V \Lambda V^T$
- V : eigenvectors (principal components)
- Λ : eigenvalues (variance explained)

4. Project onto top k eigenvectors:

- Sort by eigenvalues (descending)
- Keep top k : V_k
- Project: $X_{reduced} = \tilde{X} V_k$

Variance explained: $\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import numpy as np
import matplotlib.pyplot as plt

# Assume we have word embeddings: shape (vocab_size, 300)
# For example, Word2Vec embeddings

# Standardize the data (optional but recommended)
scaler = StandardScaler()
embeddings_scaled = scaler.fit_transform(embeddings)

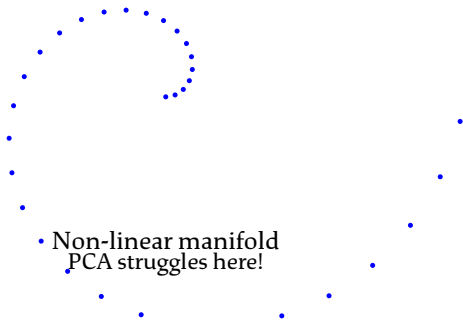
# Apply PCA
pca = PCA(n_components=2) # Reduce to 2D for visualization
embeddings_2d = pca.fit_transform(embeddings_scaled)
```

```
# Check variance explained
```

PCA Limitations

Assumes Linearity:

- Only captures linear relationships
- Word embeddings often have non-linear structure
- May miss important patterns



Other Issues:

- **Variance $\neq$ Importance:**
 - Preserves variance, not semantic structure
 - Noisy dimensions may have high variance
- **Global focus:**
 - May lose local structure
 - Clusters can become blurred
- **Interpretability:**
 - PCs are linear combinations
 - Hard to interpret semantically

When to Use PCA

- Quick first exploration
- Preprocessing for other methods

t-SNE: t-Distributed Stochastic Neighbor Embedding

Non-linear dimensionality reduction for visualization

Key Idea:

- Model similarity as probability
- High-D: Gaussian similarity
- Low-D: t-distribution similarity
- Minimize divergence between them
- Preserves **local structure**

Advantages:

- Beautiful visualizations
- Reveals clusters
- Non-linear mappings
- Great for exploration
- Widely used in practice

How it works:

1. Compute pairwise similarities in high-D:

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-||x_i - x_k||^2 / 2\sigma_i^2)}$$

2. Compute similarities in low-D using t-distribution:

$$q_{ij} = \frac{(1 + ||y_i - y_j||^2)^{-1}}{\sum_{k \neq l} (1 + ||y_k - y_l||^2)^{-1}}$$

3. Minimize KL divergence:

$$C = \sum_i KL(P_i || Q_i)$$

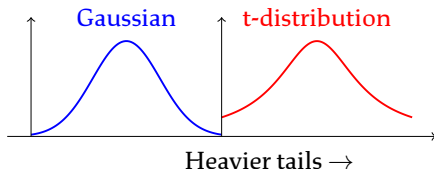
4. Use gradient descent to optimize y_i

Reference: van der Maaten & Hinton (2008). "Visualizing Data using t-SNE"

Why t-distribution in low-D?

Crowding Problem:

- High-D: many points at medium distance
- Low-D: not enough space
- Gaussian would squash together
- t-distribution has heavier tails
- Allows distant points to stay apart



Hyperparameters:

1. Perplexity (most important):

- Roughly: number of nearest neighbors
- Typical: 5-50
- Small dataset: 5-20
- Large dataset: 30-50
- Different values = different results!

2. Number of iterations:

- Minimum: 1000
- Recommended: 2000-5000
- More = better (but slower)

3. Learning rate:

Typical: 10-1000

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Apply t-SNE (can be slow for large datasets)
tsne = TSNE(
    n_components=2,          # 2D visualization
    perplexity=30,          # try 5-50
    n_iter=2000,             # iterations
    random_state=42,        # reproducibility
    verbose=1               # show progress
)

embeddings_2d = tsne.fit_transform(embeddings)

# Visualize with labels
plt.figure(figsize=(12, 10))
```

t-SNE Limitations and Caveats

1. **Slow:**

- $O(n^2)$ complexity
- Prohibitive for $>10k$ points
- Solution: Use PCA first to reduce to 50D, then t-SNE

2. **Non-deterministic:**

- Different runs $\rightarrow$ different results
- Random initialization matters
- Set random seed for reproducibility

3. **Hyperparameter sensitive:**

- Perplexity dramatically affects results
- No "correct" value
- Need to try multiple values

4. **Interpretation pitfalls:**

- Distances between clusters are meaningless!
- Only local structure is preserved
- Cluster sizes can be misleading
- Don't over-interpret global structure

5. **Not for dimensionality reduction in ML:**

- No out-of-sample extension
- Can't transform new points
- Use only for visualization!

UMAP: Uniform Manifold Approximation & Projection

Modern alternative to t-SNE (2018)

Key Advantages over t-SNE:

- **Much faster** ($O(n \log n)$ vs $O(n^2)$)
- Preserves **both** local and global structure
- Can transform new points
- Theoretically grounded (topology)
- Scales to millions of points
- More robust hyperparameters

When to Use:

- Large datasets (>10k points)
- Need global structure
- Production systems
- Consistent results important
- Most NLP visualization tasks!

How it Works:

1. Construct fuzzy topological representation of high-D data
2. Find low-D representation with similar topology
3. Use Riemannian geometry
4. Optimize cross-entropy loss

Main Hyperparameters:

1. **n_neighbors (15):**

- Local vs. global balance
- Small: local structure
- Large: global structure

2. **min_dist (0.1):**

- How tightly to pack points
- Small: tight clusters
- Large: loose, spread out

Reference: McInnes & Healy (2018). "UMAP: Uniform Manifold Approximation and Projection"

```
import umap
import matplotlib.pyplot as plt

# Apply UMAP
reducer = umap.UMAP(
    n_components=2,          # 2D visualization
    n_neighbors=15,         # local/global balance (try 5-50)
    min_dist=0.1,           # cluster tightness (try 0.0-0.99)
    metric='cosine',        # good for word embeddings
    random_state=42
)

embeddings_2d = reducer.fit_transform(embeddings)

# Can also transform new points! (unlike t-SNE)
new_embeddings_2d = reducer.transform(new_embeddings)
```

PCA vs. t-SNE vs. UMAP

Property	PCA	t-SNE	UMAP
Linear/Non-linear	Linear	Non-linear	Non-linear
Speed	Very Fast ⚡	Slow 🐌	Fast ⚡
Scalability	Excellent	Poor (<10k)	Excellent
Global structure	☐	☐	☐
Local structure	~	☐	☐
Deterministic	☐	☐	~
Out-of-sample	☐	☐	☐
Hyperparameters	Few	Sensitive	Robust
Interpretation	Medium	Hard	Medium
Best for NLP	Preprocessing	Small viz	Most tasks

Recommendations

- PCA: Quick first pass, preprocessing, need speed

Visualization Best Practices

1. **Preprocessing:**

- Standardize features (mean=0, std=1)
- Remove outliers (optional)
- For very large datasets: sample or use PCA first

2. **Try multiple hyperparameters:**

- t-SNE perplexity: 5, 10, 30, 50
- UMAP n_neighbors: 5, 15, 30, 50
- Different views reveal different structure

3. **Color intelligently:**

- By semantic category
- By cluster assignment
- By frequency (size)
- By POS tag, domain, etc.

4. **Interactive visualization:**

- Use plotly, bokeh for interactivity
- Hover tooltips with word info
- Zoom and pan
- Filter by category

5. **Annotate selectively:**

- Too many labels = clutter
- Show representative words
- Use `repel/adjust_text` to avoid overlap

Interactive Visualization

```
import plotly.express as px
import plotly.graph_objects as go
import pandas as pd

# Prepare data
df = pd.DataFrame({
    'x': embeddings_2d[:, 0],
    'y': embeddings_2d[:, 1],
    'word': words,
    'category': categories,
    'frequency': frequencies
})

# Create interactive plot with plotly
fig = px.scatter(
    df,
    x='x',
    y='y',
    color='category',
    size='frequency',
    hover_name='word',
    title='Word Embeddings Visualization',
    labels={'x': 'x', 'y': 'y', 'category': 'category', 'frequency': 'frequency'},
    style='lines')
```

1. **Embedding Quality Assessment:**

- Visualize to check if similar words cluster
- Identify anomalies and outliers
- Compare different embedding methods
- Debugging and quality control

2. **Exploratory Data Analysis:**

- Discover themes in corpus
- Identify semantic groupings
- Find polysemous words (appear in multiple clusters)
- Understand domain vocabulary

3. **Topic Modeling Visualization:**

- Visualize document embeddings
- Show topic distributions
- Interactive topic browsers (pyLDAvis, BERTopic viz)

4. **Model Interpretation:**

- Visualize attention patterns
- Show layer representations in transformers
- Understand what models learn

5. **Presentation & Communication:**

- Explain models to stakeholders
- Publication figures
- Teaching and demos

Case Study: Visualizing BERT Layers

Question: What do different BERT layers capture?

Approach:

1. Extract embeddings from each layer
2. Apply UMAP to each layer separately
3. Visualize and compare

Typical Findings:

- **Layer 0-2:** Syntactic (POS tags cluster)
- **Layer 3-8:** Semantic (meaning clusters)
- **Layer 9-12:** Task-specific

Insights:

```
from transformers import
    BertModel, BertTokenizer

model = BertModel.
    from_pretrained('bert-base-
uncased',
    output_hidden_states=True)
tokenizer = BertTokenizer.
    from_pretrained('bert-base-
uncased')

# Get embeddings from all
layers
tokens = tokenizer(sentences,
return tensors='pt', padding
```

What does a 2D visualization actually tell us about 300D space?

Consider:

- We compress 300 dimensions into 2
- Massive information loss (>99%)
- Different methods show different views
- Hyperparameters change the story

What we CAN conclude:

- Rough semantic groupings
- Relative proximities
- Cluster existence
- Outliers and anomalies
- Qualitative patterns

What we CANNOT conclude:

- Exact distances
- True high-D structure
- Cluster sizes (t-SNE)
- Between-cluster distances
- Quantitative relationships

Important

Dimensionality reduction is a **lossy transformation**. Always combine visualization with quantitative evaluation!

Practical Tips

1. Choose the right tool:

- Default: UMAP (fast, balanced)
- Need speed: PCA
- Small dataset, only local: t-SNE

2. Preprocessing matters:

- StandardScaler for PCA
- Consider normalization for UMAP/t-SNE
- Remove extreme outliers

3. Try multiple settings:

- Run with different hyperparameters
- Compare results
- Don't cherry-pick!

4. Use color wisely:

- Semantic categories
- Continuous values (frequency, sentiment)
- Consider colorblind-friendly palettes

5. Save your random seeds:

- Makes results reproducible
- Important for publications
- Helps debugging

6. Computational tips:

- For >100k points: PCA first to 50D, then UMAP
- Use `n_jobs=-1` for parallel computation
- Consider approximate nearest neighbors (Annoy, FAISS)

What we learned today:

1. **Curse of Dimensionality:** High-D space is weird, need reduction
2. **PCA:**
 - Linear, fast, preserves global structure
 - Good for preprocessing and quick exploration
3. **t-SNE:**
 - Non-linear, preserves local structure
 - Beautiful visualizations but slow
 - Sensitive to hyperparameters
4. **UMAP:**
 - Fast, scalable, balanced local+global
 - Best default choice for NLP
 - Can transform new points
5. **Best Practices:**
 - Try multiple methods and parameters
 - Use interactive visualizations
 - Don't over-interpret 2D projections

Next: Cognitive models of semantic representation!

Key References

Foundational Papers:

- Pearson, K. (1901). "On Lines and Planes of Closest Fit to Systems of Points in Space" (PCA)
- van der Maaten & Hinton (2008). "Visualizing Data using t-SNE"
- McInnes et al. (2018). "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction"

Applications:

- Jawahar et al. (2019). "What Does BERT Learn about the Structure of Language?"
- Coenen et al. (2019). "Visualizing and Measuring the Geometry of BERT"

Guides & Tutorials:

- Wattenberg et al. (2016). "How to Use t-SNE Effectively" (Distill)
- UMAP documentation: <https://umap-learn.readthedocs.io/>
- Scikit-learn User Guide: <https://scikit-learn.org/stable/modules/manifold.html>

Tools:

- Scikit-learn: PCA, t-SNE
- UMAP-learn: `pip install umap-learn`
- Plotly: `pip install plotly`

Next Lecture:

Cognitive Models of Semantic Representation

How do humans represent meaning?